《记得喊我录播》

Aop

1.哪些类需要被AOP

2.要在类中的方法的前后分别织入什么样的逻辑

得到配置类的消息，并且封装成aopConfig

```
private GPAdvisedSupport instantionAopConfig() {
  GPAopConfig config = new GPAopConfig();
  config.setPointCut(this.reader.getConfig().getProperty("pointCut"));
  config.setAspectClass(this.reader.getConfig().getProperty("aspectClass"));
  config.setAspectBefore(this.reader.getConfig().getProperty("aspectBefore"));
  config.setAspectAfter(this.reader.getConfig().getProperty("aspectAfter"));
  config.setAspectAfterThrow(this.reader.getConfig().getProperty("aspectAfterThrow"));
  config.setAspectAfterThrowingName(this.reader.getConfig().getProperty("aspectAfterThrowi
ngName"));
  return new GPAdvisedSupport(config);
}
```

会给每一个需要被拦截方法配置一个链

```
if(!(null == config.getAspectAfterThrow() || "".equals(config.getAspectAfterThrow()))){
  GPAspectJAfterThrowingAdvice advice = new
GPAspectJAfterThrowingAdvice(aspectClass.newInstance(),aspectMethods.get(config.getAspectA
fterThrow()));
  advice.setThrowName(config.getAspectAfterThrowingName());
  advices.add(advice);
}
if(!(null == config.getAspectAfter() || "".equals(config.getAspectAfter()))){
  advices.add(new
GPAfterReturningAdviceInterceptor(aspectClass.newInstance(),aspectMethods.get(config.getAs
pectAfter())));
}
if(!(null == config.getAspectBefore() || "".equals(config.getAspectBefore()))){
  advices.add(new
GPMethodBeforeAdviceInterceptor(aspectClass.newInstance(),aspectMethods.get(config.getAspe
ctBefore())));
}
methodCache.put(method,advices);
```

判断哪些类需要被代理，如果类有实现接口，就是用jdk,没有就使用cglib

```
public GPAopProxy createAopProxy(GPAdvisedSupport config) throws Exception {
  Class targetClass = config.getTargetClass();
  if(targetClass.getInterfaces().length > 0){
    return new GPJdkDynamicAopProxy(config);
  }
  return new GPCglibAopPorxy();
}
```

如何织入代码

1.首先根据方法得到我们需要执行的链（有三个，分别是前置，后置，异常）

2.定义索引=-1，然后调用前置类，直接执行before,然后递归调用proceed（）

3.索引++，得到第二个后置类，调用proceed（）(因为after在递归方法之后)

3.索引++ ，得到第三个异常类，调用proceed（）(因为只有发生异常才会被调用)

4.索引等于我们链的长度-1，就开始调用目标方法

5，执行目标方法

6.返回递归，调用after方法

7.返回递归，结束

通过索引+递归

MVC

启动容器

1.扫描所有的action

2.把请求路径，类名，方法名封装成一个handlerMapping，放入到list中

3.把handlerMapping放入到handlerAdapters map中，key为handlerMapping，value为new GPHandlerAdapter()

调用阶段

1.通过请求链接找到handlerMapping

2.通过HandlerMapping找打handlerAdapter

3.封装请求参数

4.调用方法

5.返回数据（此处有坑）

源码：

spring和springmvc他们两个容器是分开

spring:加载加了service注解的bean

springmvc：加载controller或者requestMapping这样的注解

生命周期

作业